feat: GPU native scan task — GPU-side decode of `.duckdb` block storage

:warning: DuckDB submodule change note (read first)

This branch **no longer requires the `GetDirectBlockPointer()` patch** on the DuckDB submodule (commit `cc05e7cca5`). That patch has been **reverted**; the submodule is back on upstream `cf7aeca521` (includes only the 5 small accessor patches we already carry: `GetRowGroupCollectionRef`, `GetRowGroupsDirect`, `GetColumnDirect`, `GetCompressionType`, `GetValidityData`).

The Pin-bypass now lives entirely inside Sirius via a pure-extension atomic mmap fast path (see `src/op/scan/direct_block_scan.cpp`). It matches the patched-DuckDB version within **~2–3%** on all benchmarks we ran. If the DuckDB team ever accepts the `GetDirectBlockPointer()` ask, we can swap back in a 1-line change; until then this PR doesn't ask them for anything beyond the accessors they've already seen.

See the "mmap bypass: pure Sirius" section below for the design.

Summary

Replaces DuckDB's CPU `duckdb_scan_task` table function with a Sirius-native scan for block-backed `.duckdb` storage. The extension now decodes DuckDB's on-disk compressed blocks (bitpacking, dictionary, FSST, RLE, constant, uncompressed) directly on the GPU, parsing block metadata on the device and skipping the buffer-manager Pin path for read-only databases.

Terminology: *warm* = iter-2 in same session (DuckDB buffer pool warm, OS page cache hot, GPU pool allocated). *Cold* = first query in a fresh DuckDB process; *disk-cold* = caches dropped before each query.

Warm results:

Workload	`dev` (pre-branch)	This PR	Δ
TPC-H SF=10 GPU (RTX 6000)	**26.15s** (via `duckdb_scan_task`)	**5.37s**	**4.9× (−79%)**
TPC-H SF=100 GPU (GH200)	8.4s (intra-branch)	**5.33s**	−37%; **1.33× vs CPU**
ClickBench 10M GPU (RTX 6000)	3.37s (intra-branch)	**2.10s**	−38%
ClickBench 100-shard GPU (GH200)	OOM / 29.3s	**5.00s**	−83%

Dev Q1 6.80s → ~0.25s (**27×**), Q19 7.14s → ~0.25s (**28×**). GPU wins on 15/22 TPC-H queries at SF=100 on GH200.

Cold results:

Workload	Cold (per-query avg)	Warm (suite)	Notes
TPC-H SF = 100 GH200, lazy mmap (default)	**2.4s/q** (~52.8s suite)	5.33s	GH200 auto-detect, commit `27b049e`
TPC-H SF = 100 GH200, MADV_POPULATE_READ (old)	16.4s/q	—	faulted whole DB, polluted OS cache
TPC-H SF = 100 GH200, no mmap (`Pin()`)	1.1s/q	—	lowest cold, but warm regresses
ClickBench 100-shard GH200, lazy mmap	**3.3s/q**	5.00s	
ClickBench 10M RTX6000, lazy mmap	**2.94s**	2.10s	was 12.80s with prefault
TPC-H SF = 10 RTX6000 iter 1 (buffer-cold)	**6.94s**	5.37s	pool fills during first pass

6.9× GH200 cold win from dropping `MADV_POPULATE_READ` — the prefault faulted the entire DB file on first mmap for zero benefit on ATS. Warm is unchanged.

Cold-start tiers on Q1 lineitem (SF = 10 RTX6000): **disk-cold 3.78s → buffer-cold 0.93s → warm 0.26s**. DuckDB CPU has the same 29× cold-to-warm ratio — not Sirius-specific.

What's in this PR

1. `gpu_native_scan_task` (`src/op/scan/`, `src/creator/`)

Replaces the DuckDB table-function scan path for `.duckdb` block storage. Walks `RowGroupCollection → RowGroup → ColumnData → ColumnSegment`, classifies each segment by compression type, and dispatches to GPU decode kernels. Keeps DuckDB's table-function path for parquet / non-block sources (the fallback is in `src/fallback.cpp`, unchanged).

2. GPU decode kernels (`src/cuda/scan/`)

Per compression type:

- **Bitpacking** (int8 / int16 / int32 / int64) with fused GPU-side metadata parse; single kernel per batch of segments instead of one per segment
- **Dictionary** (including string dicts); vectorized gather
- **FSST**: three-phase chunked decode (offsets, prefix-scan, byte-emit) with per-phase kernel fusion
- **RLE**, **Constant**, **Uncompressed** (int + string)

Net decode time for TPC-H SF = 100 warm Q1 is 7ms (1.1% of the 630ms wall clock). We are no longer the bottleneck.

3. Parallel scan task scheduling

Scan launches N concurrent tasks (one per hardware thread up to the cudf decode limit),

each processing disjoint row-group segments. This saturates the extension's scan thread pool and is the single biggest win at SF = 10.

4. Batched two-pass string decode

Eliminates per-segment sync + kernel launch in the string decode path by upper-bounding char allocation in pass 1 (device-side), then running a single gather in pass 2.

5. mmap bypass: pure Sirius

Read-only `.duckdb` files get mmap'd and the extension indexes block data directly, bypassing `BufferManager::Pin()` (which was 470ms / 74.6% of Q1 SF = 100 wall). **Pure extension code — no DuckDB patch needed.**

Design:

```
// single-slot atomic hot path (no mutex on the common case)
static std::atomic<mmap_file_state*> g_mmap_last{nullptr};

// keyed by BlockManager*; init under mutex, then published via atomic
static std::mutex g_mmap_mutex;
static std::unordered_map<BlockManager*, unique_ptr<mmap_file_state>> g

try_get_mmap_for_table(storage):
  bm = &storage.GetAttached().GetStorageManager().GetBlockManager();
  last = g_mmap_last.load(acquire);
  if (last && last->owner == bm) return last;   // hot path, no allocat
  // slow path: lock → find or insert → mmap file → store->g_mmap_last
```

Block offset math: `BLOCK_START = 4096*3 + block_id * alloc_size + header`. This is validated once per file via `std::call_once` against a `Pin()`'d reference block — if DuckDB's layout changes, we detect and fall back cleanly. `alloc_size` and `header_size` are read from `ColumnSegment::block->GetBlockAllocSize()` / `GetBlockHeaderSize()` so they track upstream changes automatically.

Gated on:

- `storage_manager.options.read_only == true`
- `!encryption_enabled`
- `.duckdb` single-file block manager (not parquet / other extensions)

Any mismatch falls back to `Pin()`.

6. Unified-memory auto-detect (GH200)

On GH200 with ATS, prefaulting the mmap with `MADV_POPULATE_READ` hurts perf (cold: 16.4s → 2.4s avg per TPC-H SF100 query when we skip prefault). `direct_block_scan.cpp` now auto-detects unified memory and skips prefault. PCIe path (RTX 6000) already benefits from lazy faults — prefault was removed in commit `938898f`.

7. Miscellaneous correctness fixes

- `fix: 3-word unpack for int64 bitpacking` — timestamp decode corruption

- `fix: correct chunked string decode indexing` — chunks re-read segment start
- `fix: restore stream.synchronize() between pipeline operators` — cuDF operators were seeing pre-scan stale data in rare races
- `fix: rewrite mmap bypass with thread-safe init` — earlier double-init race under contention

Rejected approaches

Tried	Why reverted
H2D / compute interleaving via dual CUDA streams + ring buffer	Net regression at SF = 10 on RTX 6000 (commit `e3e2719`). Scan bandwidth is saturated already; the extra stream synchronization cost more than the overlap gained. Worth revisiting on GH200 after other wins land.
`cudaMemcpyBatchAsync` for block transfers	~20% slower than per-segment async copies. Observed 44µs inter-op gaps vs 1µs for naive per-segment path.
`MADV_POPULATE_READ` prefault (everywhere)	Hurts GH200 cold by 6.9× (16.4s → 2.4s/query). Now gated to PCIe-only.
DuckDB `GetDirectBlockPointer()` submodule patch	Works, but adds fork maintenance cost and we don't need it — pure-Sirius atomic matches perf within 2–3%. Reverted this PR. Left as Ask #2 to the DuckDB team if they want to land it upstream.

Where to pick this up

Rough priority order for whoever takes the next sprint:

1. **cuDF internals (89% of SF = 100 Q1 GPU time)** — `hash_group_by` and CUB building blocks dominate. Profile + file issues upstream or patch cuDF directly.
2. **Ring-buffer H2D / compute overlap (GH200-only)** — skipped on PCIe; may be worth ~10% on GH200 where H2D isn't the bottleneck.
3. **GPU cache of decoded cudf columns** — only if we see repeated-query workloads on the same table (today's warm pin is 1.5ms once DuckDB's buffer pool warms; cache wins if we want to skip Pin entirely).
4. **SF = 1000 GH200 run** — we don't have the data; theory says we continue to pull ahead of CPU. Block on data generation.
5. **Bigger-than-memory on GH200** — confident via ATS theory, no empirical run. Block on cache-flush methodology.
6. **ClickBench wide-table regression** — CPU still wins ClickBench 100-shard GH200 (5.00s vs 2.54s). Not our target workload; low priority unless we decide to pursue.

Test plan

- [x] `make test` — all SQLLogicTests pass
- [x] `sirius_unittest` — unit tests pass

- [x] TPC-H SF = 10 correctness: all 22 queries return DuckDB-CPU-identical results
- [x] TPC-H SF = 100 correctness (GH200): all 22 queries return CPU-identical results
- [x] ClickBench 10M correctness: all 29 non-skipped queries match CPU
- [x] ClickBench 100-shard correctness (GH200): 16 tested queries match CPU
- [x] 2-pass benchmark after DuckDB submodule revert: no regression (TPC-H SF10 warm 5.56s, ClickBench 10M warm 1.74s)
- [x] Works with `read_only=false` (falls back to `Pin()`)
- [x] Works with encrypted DB (falls back to `Pin()`)
- [x] Works with parquet sources (unchanged fallback path)

Reviewer notes

- The 5 DuckDB accessor patches remain in the submodule. The "Ask the DuckDB team" Slack message (handoff doc B) covers upstreaming them.
- If you see perf numbers in older docs that reference the Pin-bypass via `GetDirectBlockPointer()`, those are from commit `f516a78` (now reverted). Current perf is the pure-Sirius path and matches within noise.